

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



Título del Proyecto:

MediLogic

PONDERACIÓN: 23

Horas Aproximadas: 50

Contenido

1. Resumen Ejecutivo	3
2. Competencias que desarrollaremos	3
3. Objetivos del Aprendizaje.....	3
3.1 Objetivo General.....	3
3.2 Objetivos Específicos	3
4. Enunciado del Proyecto	4
4.1 Descripción del problema a resolver.....	4
4.2 Alcance del proyecto	4
4.3 Requerimientos técnicos	7
4.4 Entregables	8
5. Metodología	9
6. Desarrollo de Habilidades Blandas	10
6.1 Proyectos en Grupo.....	10
6.2 Proyectos Individuales.....	11
7. Cronograma.....	11
8. Rúbrica de Calificación	12
8.1 Requisitos para optar a la calificación.....	12
8.2 Resumen de Puntuaciones.....	13
8.3 Detalle de la Calificación	13
8.4 Valores	15

1. Resumen Ejecutivo

Este proyecto consiste en el diseño e implementación de un sistema experto inteligente basado en lógica computacional, orientado al diagnóstico médico preliminar. El sistema, denominado MediLogic, permitirá a los usuarios (pacientes) ingresar sus síntomas, enfermedades crónicas y alergias a medicamentos para recibir un informe con posibles enfermedades, porcentaje de afinidad y tratamiento sugerido. Utilizando reglas definidas en Tau-Prolog, el sistema realizará inferencias que relacionen los datos ingresados con una base de conocimientos médica estructurada. Esta iniciativa busca potenciar el razonamiento computacional de los estudiantes mediante la resolución de problemas del mundo real, y su aplicación podría extenderse a contextos de salud pública o telemedicina.

2. Competencias que desarrollaremos

- Integra fundamentos de lógica, realidad aumentada y simulación robótica mediante el uso de entornos de desarrollo como Tau Prolog, AR.js y Three.js para resolver problemas relacionados a inferencia, interacción digital y simulación de comportamientos autónomo.
- Implementa soluciones de inteligencia artificial empleando técnicas de inferencia lógica, diseño de entornos virtuales y control de robots en distintos escenarios de optimización, clasificación y toma de decisiones.
- Desarrolla hechos, reglas, expresiones y predicados recursivos mediante el uso de cláusulas, ciclos, listas, unificación y cortes en Tau Prolog para modelar bases de conocimiento y resolver problemas de inferencia lógica.

3. Objetivos del Aprendizaje

3.1 Objetivo General

Al finalizar el proyecto, los estudiantes serán capaces de desarrollar una solución tecnológica que resuelva un problema real o simulado por medio de sistemas inteligentes, utilizando Prolog como el lenguaje y principal herramienta para el desarrollo de la lógica e inferencia. Los estudiantes aplicarán habilidades de programación, diseño de sistemas, trabajo en equipo y metodología de desarrollo, para entregar un prototipo funcional junto con su documentación técnica.

3.2 Objetivos Específicos

Al finalizar el proyecto, los estudiantes deberán ser capaces de:

1. **Desarrollar un sistema funcional:** Aplicar los conocimientos adquiridos en el curso para diseñar, implementar y probar un sistema experto capaz de inferir enfermedades

y sugerir medicamentos seguros a partir de los datos ingresados por el usuario. Esto incluye el uso de HTML, CSS, JavaScript puro y Tau-Prolog.

2. **Diseñar interfaces de usuario funcionales y amigables:** Implementar interfaces intuitivas que permitan a los usuarios ingresar información clínica de manera clara, y consultar los diagnósticos generados de forma comprensible y visual.
3. **Documentar correctamente el desarrollo del sistema:** Elaborar la documentación técnica del proyecto, incluyendo manual de usuario, guías de instalación, estructura del archivo Prolog, pruebas realizadas y justificación de reglas activadas en el diagnóstico.

4. Enunciado del Proyecto

4.1 Descripción del problema a resolver

La falta de acceso inmediato a un diagnóstico médico preliminar, combinada con la presencia de enfermedades crónicas y alergias, representa un reto significativo en la atención sanitaria básica. Muchos pacientes no pueden identificar correctamente sus padecimientos ni saben qué tratamientos pueden o no utilizar. Esto puede derivar en complicaciones, automedicación riesgosa o tratamientos inadecuados.

Este proyecto propone el diseño e implementación de un sistema experto que, con base en lógica computacional, pueda analizar los síntomas, alergias y enfermedades preexistentes del usuario para generar un informe con posibles diagnósticos, el grado de afinidad con cada uno, y medicamentos recomendados evitando contraindicaciones.

El sistema será construido sobre un motor lógico desarrollado en Tau-Prolog, el cual, mediante reglas lógicas codificadas en un archivo .pl, podrá realizar inferencias sobre los datos ingresados por el usuario. Se busca que el sistema sirva como herramienta de orientación médica y educación, simulando el razonamiento de un especialista de salud a través de programación lógica.

4.2 Alcance del proyecto

El sistema inteligente médico por desarrollar, denominado MediLogic, estará compuesto por dos módulos principales: uno destinado a los usuarios pacientes y otro reservado para administradores o personal médico autorizado. Ambos módulos deberán estar integrados en una interfaz web funcional, clara e intuitiva, desarrollada con tecnologías básicas del lado del cliente (HTML, CSS y JavaScript), junto con una lógica de inferencia implementada íntegramente en Tau-Prolog.

Inicio

Al ingresar al sistema, los usuarios encontrarán una pantalla principal accesible sin autenticación. En ella se mostrará una descripción general del sistema y su funcionalidad,

explicando que se trata de una herramienta de apoyo diagnóstico preliminar que no sustituye la consulta médica profesional. Desde esta pantalla, el usuario podrá elegir entre acceder al módulo de diagnóstico para pacientes o al módulo administrativo para personal médico, este último protegido mediante credenciales.

Pacientes

El módulo destinado a los usuarios pacientes permitirá ingresar información clínica básica a través de formularios interactivos. El sistema debe permitir seleccionar síntomas presentes a través de checkboxes, así como registrar alergias a medicamentos y enfermedades crónicas preexistentes como diabetes, hipertensión, enfermedades autoinmunes, entre otras.

Además, para mejorar la precisión del análisis, el sistema deberá permitir que el usuario indique el nivel de severidad de cada síntoma (leve, moderado, severo), lo cual debe influir en el cálculo de la afinidad con las enfermedades registradas.

Una vez completado el ingreso de datos, el usuario podrá solicitar un análisis. El sistema, utilizando la base de conocimiento en Tau-Prolog, realizará un proceso de inferencia lógica para determinar posibles enfermedades asociadas a los síntomas proporcionados. Estas enfermedades deberán ordenarse por porcentaje de coincidencia, expresado como nivel de afinidad entre el perfil del paciente y las características clínicas de cada enfermedad.

Además del diagnóstico, el sistema debe sugerir medicamentos adecuados para tratar cada una de las enfermedades listadas, siempre y cuando no estén contraindicados por las alergias o condiciones crónicas indicadas. En caso de conflictos entre tratamientos posibles y enfermedades coexistentes, el sistema deberá identificar estas incompatibilidades y excluir opciones que puedan representar un riesgo para el paciente, sugiriendo otro posible medicamento con el mismo fin, pero sin repercusiones para su estado de salud general.

Finalmente, el sistema deberá emitir una recomendación de acción para el usuario, indicando el nivel de urgencia con frases como: "Consulta médica inmediata sugerida", "Posible automanejo" u "Observación recomendada".

El informe que se genere debe incluir:

- Una lista de enfermedades sugeridas, ordenadas de mayor a menor probabilidad.
- El porcentaje de afinidad correspondiente a cada diagnóstico.
- El medicamento más seguro y efectivo propuesto, evitando riesgos por alergias o interacciones negativas.
- Una explicación detallada que indique qué reglas Prolog se activaron para llegar a las conclusiones presentadas.
- Un nivel de urgencia estimado para cada diagnóstico, con base en los síntomas y su severidad.
- Visualización clara de los resultados mediante tablas, secciones y gráficos explicativos como barras de afinidad o alertas de advertencia.

Adicionalmente, el sistema deberá mantener un historial local temporal de diagnósticos durante la sesión activa en el navegador, permitiendo que el paciente revise análisis anteriores mientras navega.

También deberá ofrecer la opción de descargar el informe en formato PDF, incluyendo la fecha, un resumen del diagnóstico, las advertencias importantes, los medicamentos sugeridos, las reglas activadas y el sello visual del sistema.

Administrador

El módulo administrativo estará protegido por autenticación y será accesible únicamente para usuarios con credenciales válidas. Este módulo ofrecerá un panel de control completo que permita gestionar la base de conocimientos del sistema en tiempo real, mediante formularios o interfaces estructuradas que no requieran editar manualmente el archivo .pl.

Desde este panel, el administrador podrá realizar las siguientes acciones:

- Crear, editar y eliminar enfermedades registradas en el sistema, incluyendo su nombre, descripción, síntomas asociados y medicamentos contraindicados.
- Registrar nuevos síntomas y asociarlos a enfermedades específicas mediante criterios de compatibilidad.
- Administrar medicamentos disponibles, indicando qué enfermedades pueden tratar y qué contraindicaciones presentan.
- Definir relaciones de contraindicación entre medicamentos, enfermedades crónicas y alergias.
- Clasificar enfermedades por sistema del cuerpo (respiratorio, digestivo, endocrino, etc.) o por tipo (viral, crónico, inmunológico), para facilitar su administración y consulta.
- Visualizar y exportar el archivo .pl actual con toda la base lógica del sistema.
- Cargar un nuevo archivo .pl para actualizar el sistema sin necesidad de reprogramar la interfaz.

Todos los cambios realizados mediante la interfaz administrativa deben reflejarse automáticamente en el archivo Prolog, asegurando así la persistencia, actualización y validez de las reglas y hechos registrados. Esta integración entre el frontend del sistema y la lógica declarativa debe garantizar que el motor de inferencia pueda utilizar los datos actualizados de forma inmediata.

4.3 Requerimientos técnicos

Para el desarrollo del proyecto MediLogic, los estudiantes deberán emplear exclusivamente tecnologías del lado del cliente, sin el uso de servidores ni frameworks avanzados. El objetivo es que los estudiantes comprendan, a profundidad, los fundamentos de la inteligencia artificial simbólica aplicada a través de un sistema web ligero y funcional.

Las herramientas y tecnologías requeridas son:

- **Lenguajes de programación:**

El sistema debe ser desarrollado utilizando únicamente HTML, CSS y JavaScript puro. No se permite el uso de frameworks modernos como React, Angular o Vue, ni runtimes como Node.js. Esta restricción busca fortalecer las habilidades base en el desarrollo web y permitir la integración directa con el motor lógico Tau-Prolog.

- **Motor lógico:**

Toda la lógica del sistema deberá estar implementada en Tau-Prolog, ejecutándose en el entorno del navegador a través de su versión JavaScript. La base de conocimiento debe estar contenida exclusivamente en un archivo .pl, el cual incluirá hechos, reglas, y estructuras que representen los síntomas, enfermedades, medicamentos y sus relaciones lógicas.

- **Interfaz web:**

Se espera una interfaz clara, amigable y accesible que permita a los usuarios interactuar con el sistema sin necesidad de conocimientos técnicos. Esta deberá facilitar el ingreso de información, el análisis automático y la visualización de los resultados generados por el motor lógico.

- **Control de versiones:**

Todo el código fuente, documentación técnica y recursos del proyecto deberán alojarse en un repositorio de GitHub privado, cuyo nombre será: IA1_1S2025_P1_G# (sustituyendo # por el número correspondiente al grupo). El auxiliar robinbuezo11 deberá ser agregado como colaborador del repositorio para su evaluación.

- **Licenciamiento:**

El proyecto deberá tener una **licencia de software MIT**, claramente indicada en el repositorio. Una vez calificado, el repositorio deberá hacerse público para fomentar el acceso abierto y la reutilización académica del código.

4.4 Entregables

Tipo	Descripción
Aplicación Web	Enlace funcional a la aplicación web publicada mediante GitHub Pages . Debe estar alojada dentro del mismo repositorio del proyecto.
Manual Técnico	Documento en PDF que incluya una explicación completa de la arquitectura del sistema, las herramientas utilizadas, la estructura del archivo .pl, el flujo de interacción entre los módulos, y los procesos lógicos implementados. También debe incluir una descripción detallada de las reglas definidas en Tau-Prolog y su justificación, así como las decisiones tomadas en la construcción del sistema.
Manual de Usuario	Documento destinado a los usuarios finales del sistema. Debe contener instrucciones paso a paso sobre el uso del módulo de pacientes y el módulo administrativo, incluyendo capturas de pantalla, posibles errores comunes, recomendaciones y advertencias. También debe explicar cómo interpretar los resultados presentados por el sistema y cómo utilizar el informe descargable en PDF.
Código Fuente	Repositorio de GitHub que contenga el código completo del proyecto: archivos HTML, CSS, JavaScript y el archivo .pl con la base lógica en Tau-Prolog. El código debe estar organizado, debidamente comentado, y reflejar buenas prácticas de desarrollo y control de versiones. El repositorio debe mantenerse privado hasta su revisión, y hacerse público después de su evaluación.
Evaluación de coherencia diagnóstica	Documento adicional en el que se presenten al menos tres casos clínicos completos con datos de entrada (síntomas, alergias y enfermedades crónicas) y un análisis del resultado obtenido por el sistema. El estudiante deberá explicar si el diagnóstico fue coherente o no, justificando su respuesta. Este entregable fortalecerá la comprensión del modelo lógico y fomentará el pensamiento crítico.

- **Fecha límite de entrega:** Pendiente.
- **Colaborador obligatorio en el repositorio:** El auxiliar *robinbuezo11* debe ser agregado al repositorio como colaborador con permisos de lectura completa.
- **Nombre del repositorio:** Pendiente. Agregar la carpeta llamada "Proyecto1"
- **Medio de entrega:** El enlace al repositorio y al sitio web publicado debe ser enviado a través de la plataforma UEDI

5. Metodología

El desarrollo del proyecto MediLogic deberá seguir una metodología estructurada que permita a los estudiantes organizar, ejecutar y documentar cada etapa de forma efectiva. Se recomienda la aplicación de metodologías ágiles, como SCRUM, para la gestión del tiempo, la colaboración en equipo y la entrega progresiva de avances.

A continuación, se describen las fases mínimas que los estudiantes deberán seguir durante el ciclo de desarrollo del proyecto:

- 1. Investigación preliminar:** Los estudiantes deben realizar una revisión técnica sobre sistemas expertos médicos, lógica declarativa, motores de inferencia como Tau-Prolog, y ejemplos de diagnóstico automatizado. También deberán estudiar las implicaciones de la sintaxis Prolog, su integración en JavaScript y buenas prácticas en el diseño de reglas.
- 2. Diseño del sistema:** Se deben elaborar diagramas de flujo, maquetas de interfaz (mockups) y una representación inicial de la estructura del archivo .pl. En esta fase se definirán los síntomas, enfermedades, medicamentos, contraindicaciones y las reglas básicas de inferencia.
- 3. Construcción de la interfaz web:** Se desarrollará la aplicación utilizando exclusivamente HTML, CSS y JavaScript puro. Se implementará la navegación entre módulos, la captura de información del paciente, la comunicación con el motor Prolog y la presentación del informe de diagnóstico.
- 4. Implementación del motor lógico:** Los estudiantes deberán desarrollar el archivo .pl que contendrá la base de conocimiento médica (hechos y reglas). Deberán aplicar lógica de predicados para vincular síntomas con enfermedades, validar contraindicaciones, y calcular el porcentaje de afinidad entre perfiles clínicos y diagnósticos.
- 5. Integración del sistema:** Se conectará la interfaz desarrollada con el motor lógico, garantizando el flujo correcto de datos desde el ingreso de información hasta la generación del resultado. También se integrará el módulo administrativo, que permitirá modificar la base de conocimientos desde la interfaz sin intervenir directamente el archivo .pl.
- 6. Pruebas funcionales y validación:** Se realizarán pruebas sistemáticas utilizando casos clínicos simulados para verificar el funcionamiento del sistema, la coherencia de los resultados y la activación correcta de las reglas. Esta fase también contempla el desarrollo del entregable de evaluación de coherencia diagnóstica, con al menos tres casos completos documentados.
- 7. Documentación:** Los estudiantes deberán preparar el manual de usuario, el manual técnico, y el informe general del proyecto, detallando la estructura del sistema, la justificación de sus decisiones lógicas y tecnológicas, así como las dificultades enfrentadas durante el desarrollo.

8. **Presentación final:** Finalmente, los equipos estudiantes su proyecto, demostrando su funcionamiento, explicando las reglas utilizadas y presentando los entregables requeridos. Se evaluará tanto la calidad técnica como la claridad en la presentación y defensa del trabajo.

6. Desarrollo de Habilidades Blandas

Para complementar el desarrollo técnico del sistema *MediLogic*, los estudiantes deberán fortalecer una serie de habilidades blandas que son fundamentales en el ámbito profesional. Estas habilidades permitirán mejorar la comunicación, la colaboración, el liderazgo y la capacidad de resolver problemas en entornos reales o simulados de desarrollo de software.

6.1 Proyectos en Grupo

El proyecto está diseñado para ser trabajado en equipos. La colaboración efectiva será clave para lograr un producto funcional y bien documentado. Cada equipo deberá organizarse internamente, asignar roles específicos (como líder de proyecto, programador, diseñador de interfaz, responsable de documentación, entre otros) y utilizar herramientas colaborativas que les permitan coordinar sus tareas, compartir avances y mantenerse organizados.

Herramientas sugeridas para la gestión de equipo:

- **Trello o Notion** para asignación de tareas y seguimiento de sprints.
- **Google Docs/Sheets** para documentación compartida.
- **GitHub** para control de versiones y revisión colaborativa de código.

6.1.1 Trabajo en Equipo

Cada grupo debe dividir las responsabilidades de forma estratégica, respetando las habilidades y fortalezas de sus integrantes. Todos los miembros deben contribuir activamente en cada etapa del proyecto y cumplir con los plazos establecidos, promoviendo un ambiente de trabajo cooperativo y profesional.

6.1.2 Comunicación Efectiva

Los equipos deberán realizar presentaciones periódicas (según lo defina el auxiliar), compartir avances, discutir bloqueos técnicos y recibir retroalimentación para mejorar continuamente. La comunicación clara y respetuosa será clave para la toma de decisiones y resolución de problemas grupales.

6.1.3 Resolución de Conflictos

Durante el desarrollo pueden surgir desacuerdos relacionados con decisiones técnicas, diseño o distribución de tareas. Los estudiantes deberán aplicar estrategias de resolución de conflictos, mediación y negociación para asegurar la continuidad del trabajo sin comprometer la cohesión del equipo.

6.2 Proyectos Individuales

En el caso de estudiantes autorizados a trabajar individualmente (por aprobación del docente auxiliar), el proyecto representa una oportunidad para desarrollar autonomía y autoevaluación. Cada estudiante asumirá la totalidad de las fases del proyecto: investigación, desarrollo, pruebas y documentación.

6.2.1 Autogestión del Tiempo

El estudiante deberá planificar sus tiempos de trabajo, dividir sus actividades por fases y cumplir con las fechas de entrega. Se espera el uso de cronogramas personales y técnicas de gestión de tareas como Pomodoro o Kanban.

6.2.2 Responsabilidad y Compromiso

Al trabajar de forma independiente, el estudiante asume completa responsabilidad por la calidad y completitud del proyecto. Esto fomenta el compromiso con el aprendizaje autónomo, la ética profesional y la entrega de productos funcionales.

6.2.3 Resolución de Problemas

El proyecto individual exige que el estudiante resuelva desafíos técnicos por su cuenta, lo que impulsa la creatividad, la investigación y la toma de decisiones lógica ante errores o problemas emergentes durante el desarrollo.

6.2.4 Reflexión Personal

Al finalizar el proyecto, el estudiante deberá realizar una autoevaluación crítica en la que reflexione sobre lo aprendido, los retos enfrentados y las habilidades que necesita mejorar. Esta práctica promueve la mejora continua y la consolidación del conocimiento adquirido.

7. Cronograma

El desarrollo del proyecto MediLogic se organizará en fases clave para asegurar un progreso continuo y bien estructurado. Se realizarán dos entregas: la primera servirá para validar el avance inicial del sistema (estructura base, lógica preliminar y diseño), y la segunda corresponderá al proyecto completo, ya funcional y documentado.

Tipo	Fecha Inicio	Fecha Fin
Asignación de Proyecto		
Entrega No. 1 – Tarea 1		
Entrega No. 2 – Proyecto Final		
Calificación		

Detalle por entrega

- **Entrega No. 1 – Tarea 1:** Esta entrega servirá para evaluar el avance inicial del proyecto. Se espera que los estudiantes presenten una versión preliminar del sistema, que incluya el diseño de la interfaz, el archivo .pl con hechos básicos y algunas reglas ya funcionales, así como la estructura del repositorio GitHub y documentación inicial (borrador del manual técnico o de usuario).
- **Entrega No. 2 – Proyecto Final:** Corresponde a la versión final del sistema MediLogic, completamente funcional, con todos los módulos implementados, pruebas realizadas, documentación completa y publicación del sitio en GitHub Pages.

8. Rúbrica de Calificación

8.1 Requisitos para optar a la calificación

Antes de ser evaluado, el proyecto deberá cumplir con los siguientes requisitos mínimos. Si alguno de estos no se cumple, el proyecto no podrá ser calificado y se considerará como no entregado o incompleto:

Tema	Descripción	Cumple (Sí/No)
Cumplimiento de la tecnología establecida	El desarrollo debe haberse realizado utilizando únicamente HTML, CSS, JavaScript puro y Tau-Prolog. No se permite el uso de frameworks ni runtimes externos.	
Uso de herramientas requeridas	El sistema debe estar publicado en GitHub Pages y gestionado mediante un repositorio en GitHub con control de versiones adecuado.	
Gestión y entregas del proyecto	Se deben haber entregado ambas fases (avance y entrega final) dentro de las fechas establecidas en el cronograma. Todas las versiones deben estar registradas.	
Documentación obligatoria	Se deben entregar el manual de usuario, el manual técnico, todos debidamente elaborados. El informe técnico debe contener diagramas.	
Evaluación de coherencia diagnóstica	Debe incluir al menos tres casos clínicos analizados con reflexión crítica sobre los resultados generados por el sistema.	

8.2 Resumen de Puntuaciones

Área	Puntos Totales	Puntos Obtenidos
1. Conocimiento		
Interfaz de Usuario	5	
Módulo de Paciente	50	
Modulo del Administrador	33	
Sub-Total	88	
2. Habilidades		
Documentación	6	
Preguntas	6	
Sub-Total	12	
TOTAL	100	

8.3 Detalle de la Calificación

Descripción de Ponderación	Valor	Observación	Punteo
1. Conocimiento			
Interfaz de Usuario	5	Interfaz clara, amigable y apegada a los requerimientos	
Módulo del Paciente	50		
Formulario para ingreso de síntomas, enfermedades crónicas y alergias	5	Completo, validado y funcional	
Registro de severidad de síntomas e impacto en el diagnóstico	5	Interfaz implementada correctamente	
Hechos en Prolog de enfermedades, síntomas y medicamentos	5	Definidos correctamente en el archivo .pl	
Reglas en Prolog para diagnóstico basado en afinidad y contraindicaciones	10	Precisas, funcionales y bien estructuradas	
Presentación lógica y gráfica del informe de diagnóstico	10	Resultados claros, ordenados y visuales	

Generación de nivel de urgencia según perfil clínico (leve/moderado/severo)	5	Diagnóstico incluye recomendación de acción	
Sugerencia segura de medicamentos evitando conflictos con alergias/enfermedades	10	Valida condiciones correctamente	
Módulo del Administrador	33		
Gestión de enfermedades, síntomas y clasificación por sistemas o tipo	10	Crear, editar, eliminar correctamente	
Gestión de medicamentos y definición de contraindicaciones	10	Interfaz funcional y reglas aplicadas	
Gestión del archivo .pl (carga y descarga, integración lógica en tiempo real)	8	Sin errores, lectura y escritura funcional	
Interfaz de administración funcional y comprensible	5	Fluida, ordenada y alineada con requerimientos	
2. Habilidades	12		
Documentación	6		
Manual Técnico (estructura, reglas, uso de Prolog, decisiones de diseño)	2	Completo y detallado	
Manual de Usuario (paso a paso, capturas, explicación de resultados)	2	Claro y orientado al usuario final	
Evaluación de coherencia diagnóstica (3 casos clínicos analizados y justificados)	2	Evaluación técnica y crítica bien documentada	
Preguntas	6		
Pregunta 1	2	Pregunta teórica o sobre el desarrollo del proyecto	
Pregunta 2	2	Pregunta teórica o sobre el desarrollo del proyecto	
Pregunta 3	2	Pregunta teórica o sobre el desarrollo del proyecto	
Penalizaciones			

Entrega fuera del plazo establecido	-20	Solo si se entrega fuera del plazo oficial (En caso de que la calificación sea mucho tiempo después de la entrega la penalización puede ser mayor).	
Impuntualidad en la calificación	-20	En caso de impuntualidad en la calificación.	
TOTAL	100		

8.4 Valores

En el desarrollo del proyecto MediLogic, se espera que los estudiantes demuestren un alto nivel de honestidad académica, responsabilidad ética y compromiso profesional. El cumplimiento de los principios que se detallan a continuación será obligatorio y cualquier incumplimiento será sancionado conforme a las normativas vigentes de la Escuela de Ciencias y Sistemas.

1. Originalidad del Trabajo

Cada estudiante o equipo debe desarrollar su propio código, archivo Prolog (.pl), documentación técnica y recursos asociados. Se espera que el trabajo entregado sea producto del conocimiento adquirido en el curso y del esfuerzo auténtico de sus autores.

2. Prohibición de Copias y Plagio

Queda estrictamente prohibido copiar, replicar o adaptar total o parcialmente el código, documentación, lógica en Prolog o cualquier componente del proyecto desde otras fuentes sin el debido análisis, modificación y referencia.

- La detección de plagio (entre compañeros, de internet o de ciclos anteriores) será penalizada con calificación de 0 puntos.
- Los responsables serán **reportados formalmente a la Escuela de Ciencias y Sistemas**.
- Esto incluye el uso de ediciones superficiales de código ajeno, sin comprensión real ni justificación lógica.

3. Uso Responsable de Recursos Externos

Está permitido el uso de bibliotecas de consulta, ejemplos o fragmentos educativos siempre y cuando:

- Se referencien adecuadamente en el código o en los anexos.
- Se comprenda completamente su funcionamiento.
- No se utilicen como sustituto de la lógica propia del proyecto. Cualquier duda deberá ser consultada con el catedrático o auxiliar antes de su uso.

4. **Revisión y Validación del Trabajo**

El equipo docente podrá utilizar herramientas automáticas y revisiones manuales para comparar entregas y detectar similitudes no justificadas.

- En caso de sospecha, el estudiante deberá defender su solución, explicar sus decisiones y justificar el funcionamiento de su código y archivo Prolog.
- Si no logra demostrar la autoría o comprensión del trabajo, se asignará **una calificación de 0 puntos**, sin opción a apelación académica.

5. **Licencia y Transparencia del Proyecto**

- El proyecto debe ser **OpenSource** bajo licencia **MIT**.
- Todo el código, archivos .pl, documentación técnica, pruebas y manuales deberán estar alojados en un repositorio de GitHub privado durante el desarrollo.
- Una vez calificado, el repositorio deberá hacerse público.
- El auxiliar del curso (**robinbuezo11**) debe ser agregado como colaborador en el repositorio.
- El nombre del repositorio debe seguir el formato: Pendiente. Agregar la carpeta llamada "Proyecto1"

6. **Entrega y Fechas Límite**

- No se permitirá realizar modificaciones al código ni a la documentación después de la fecha de entrega final establecida en el cronograma.
- Las entregas fuera del plazo establecido pueden considerarse no válidas y se aplicarán las penalizaciones indicadas, a menos que se haya aprobado una prórroga justificada con anterioridad.